

أولاً: توضيح سريع — ما هو Multi-threading ؟

الـ **Multi-threading** هو أسلوب في البرمجة يسمح بتقسيم تنفيذ البرنامج إلى عدة خيوط (**Threads**) تعمل بالتوازي داخل نفس العملية. (**Process**)

الهدف: تحسين الأداء واستغلال تعدد الأنوية. (**CPU cores**)

مثال:

- بدلاً من تنفيذ المهام بالتتابع (تحميل ملف ثم حفظه ثم تحليله)
- يمكن تشغيل كل مهمة في خيط منفصل بحيث تعمل كلها معاً في نفس الوقت.

⚙️ الحفاظ على الحالة (State Management)

عند استخدام الـ **multi-threading** ، كل **Thread** يمتلك مساحة ذاكرة خاصة به، مما يجعل مشاركة الحالة (**state**) بين الخيوط أمراً حساساً.

للحفاظ على التناسق، نستخدم تقنيات مثل:

- **Locks / Mutexes / Monitors** (لتجنب التعارض)
- **Immutable objects**
- **Thread-safe collections**
- **Async/Await patterns** في التطبيقات الحديثة مثل **.NET** و **Node.js**

Report on Caching Using Redis

1. Introduction

Caching is a performance optimization technique that temporarily stores frequently accessed data in memory. This allows applications to respond faster by reducing the need to repeatedly fetch data from a slow source such as a database or external API.

Redis (Remote Dictionary Server) is one of the most popular in-memory data stores used for caching. It offers high speed, scalability, and support for various data structures such as strings, hashes, lists, sets, and sorted sets.

2. Why Use Caching?

Caching helps in:

- **Reducing database load:** Frequently requested queries are served from cache.
 - **Improving response time:** Data retrieval from RAM is faster than from disk.
 - **Enhancing scalability:** Applications can handle more concurrent users.
 - **Reducing operational costs:** Fewer queries and less bandwidth consumption.
-

3. Redis Overview

Redis is an **in-memory key-value data store** that can persist data to disk if required. It supports both **synchronous** and **asynchronous replication** and offers mechanisms like:

- **TTL (Time To Live):** Automatic expiration of cache entries.
- **Pub/Sub messaging:** Communication between services.
- **Persistence options:** RDB snapshots and AOF (Append Only File).

Redis is widely used in systems built with frameworks such as ASP.NET Core, Node.js, Django, and Spring Boot.

4. Redis Architecture

Redis runs as a server that listens for client requests on a specific port (default 6379). The client connects, issues commands (GET, SET, etc.), and receives responses. Data is stored in-memory, which makes operations extremely fast (in microseconds).

You can deploy Redis in different modes:

- **Standalone** (single instance)
 - **Master-Slave replication**
 - **Cluster mode** for distributed caching across nodes
-

5. Implementing Caching with Redis

Example in ASP.NET Core:

Install Redis package:

```
dotnet add package Microsoft.Extensions.Caching.StackExchangeRedis
```

Configure in Program.cs:

```
builder.Services.AddStackExchangeRedisCache(options =>
{
    options.Configuration = "localhost:6379";
    options.InstanceName = "LearningApp_";
});
```

Use it in a service:

```
public class CourseService
{
    private readonly IDistributedCache _cache;

    public CourseService(IDistributedCache cache)
    {
        _cache = cache;
    }
}
```

```
public async Task<string> GetCourseAsync(string courseId)
{
    var cacheKey = $"course_{courseId}";
    var cached = await _cache.GetStringAsync(cacheKey);
    if (cached != null)
        return cached;

    // Simulate DB call
    var course = "AI Fundamentals";
    await _cache.SetStringAsync(cacheKey, course,
        new DistributedCacheEntryOptions
        {
            AbsoluteExpirationRelativeToNow = TimeSpan.FromMinutes(30)
        });
    return course;
}
```

6. Cache Invalidation

Cache invalidation ensures data consistency when the source data changes. Common strategies include:

- **Time-based expiration (TTL)**
 - **Manual invalidation** when updating/deleting data
 - **Versioning or tags** to identify outdated entries
-

7. Advantages of Redis Caching

- Extremely **fast** (in-memory access)
 - **Scalable** and supports clustering
 - **Flexible** data structures
 - **Persistence options** for fault tolerance
 - Supports **Pub/Sub** and **Lua scripting**
-

8. Conclusion

Redis provides a robust and efficient caching layer suitable for high-performance applications.

By storing frequently accessed data in-memory and using strategies like TTL and clustering, Redis helps reduce latency, improve throughput, and enhance scalability.

Caching with Redis is an essential technique for any developer aiming to build **responsive, reliable, and scalable** systems.